VISION-LANGUAGE MODELS

Lecture 5

Video Understanding

Outline

I **Tokenizing video** — Which frames to sample; frame vs tubelet tokens; how time enters the position encoding

II **Managing attention** — Why T·S tokens make attention quadratic-squared; factorized space-time attention; pooling and resamplers

III **Long video** — The context wall for hour-long video: keyframe selection, memory, streaming, audio & subtitles

IV **Tasks and evaluation** — Captioning, video QA, temporal grounding — and the benchmark trap: are models actually using time?

PART I

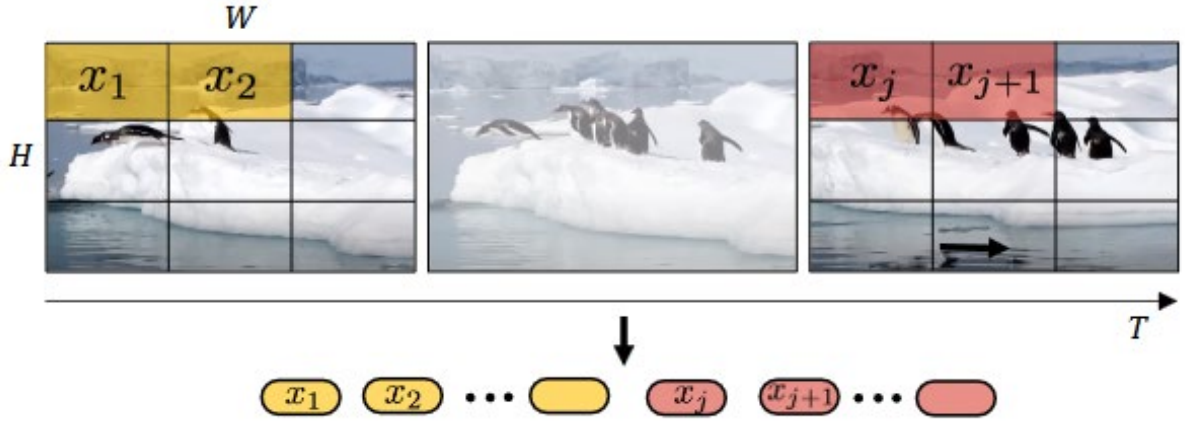
Tokenizing video

Moving from image to video

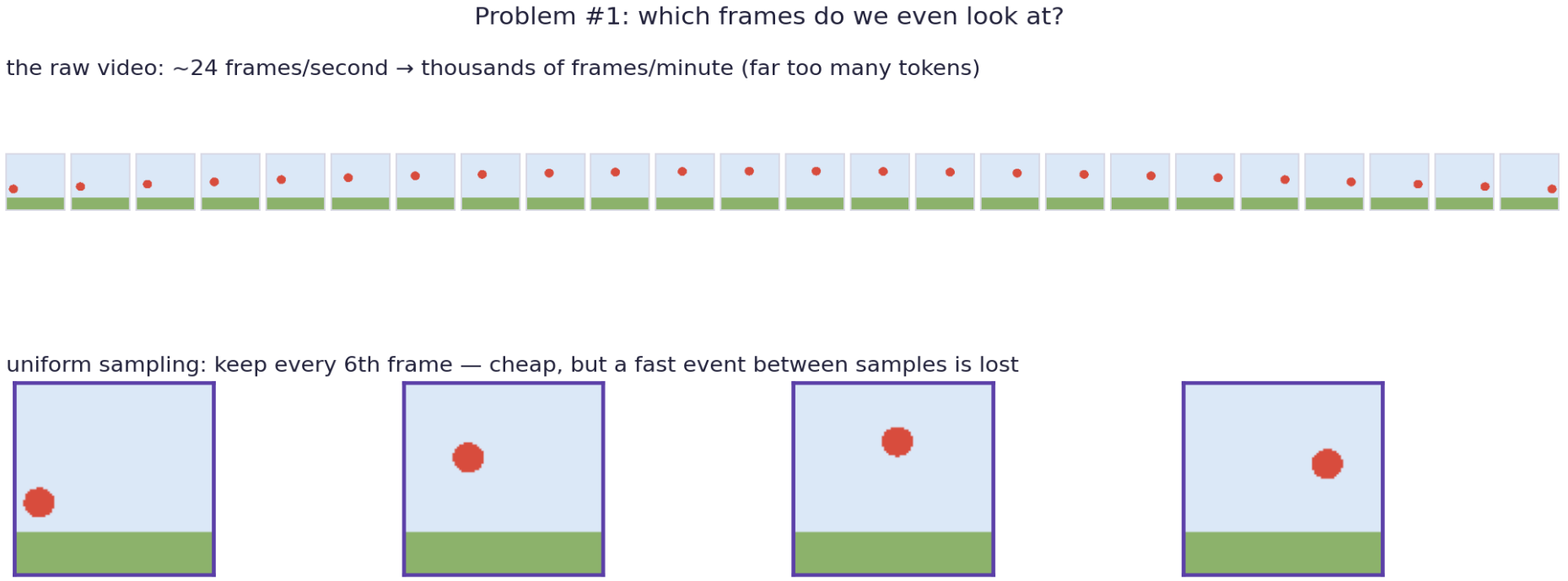
- **The image-based VLM flow:**
  - ViT tokenizes an image → CLIP aligns it with text → a bridge/projector feeds an LLM → production VLMs answers

- **But a video is "just a sequence of images" — so what changes?**
  - Scale: 1 minute at 24 fps = 1,440 frames. Naively, 1,440 × 576 tokens ≈ 830,000 tokens for one minute — impossible with quadratic attention
  - Time is a new kind of information: motion, causality, order, "before/after" — not visible in any single frame
  - New questions become possible: "What happened just before the crash?" · "Did she pick up the cup or put it down?" · "When does the goal happen?"

- Designing a practical solution requires shrinking the token stream in some way



Problem 1a: which frames do we consider?



- Common approach: input frame down-sampling
  - Uniform sampling (every k-th frame): simple, standard, but ignores scene content
    - a fast event between samples vanishes;
    - but a static scene wastes the token budget since nothing is changing but still being encoded
  - Alternatives sampling schemes:

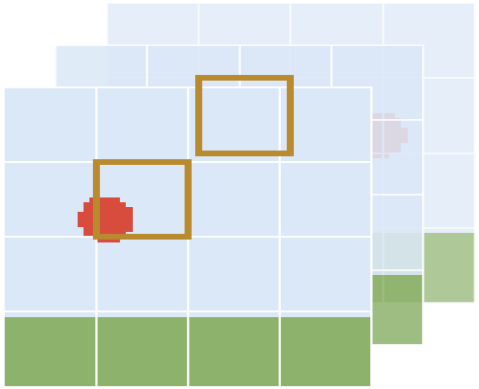
    - FPS-based (fixed frames/second, so token count scales with duration)

    - scene-change / keyframe selection, or query-aware sampling (pick frames relevant to the question)

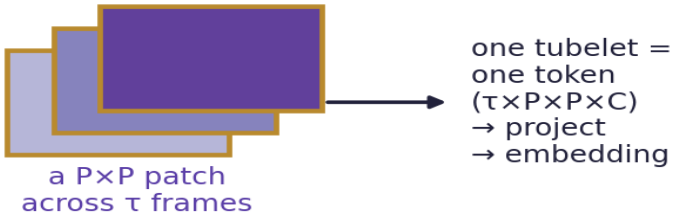
Sampling strategies after Bain et al. 2021, Lei et al. 2021 (ClipBERT).

Problem 1b: frame tokens vs tubelet tokens

From 2D patches (L1) to 3D tubelets: tokens now span space AND time



3 stacked frames, each patchified



- **Frame tokens**: patchify each sampled frame independently (as for single image), then concatenate each frames' token in order.
  - Simplest approach, time resides only in the sequence order + position encoding.
  - Not that time/frame position embedding simply added to each frame patch's positional embedding
- **Tubelet tokens**: one token spans a P×P patch across τ consecutive frames (a 3D convolution / linear projection over t×h×w).
  - injects short-term motion into each token;
  - fewer tokens for the same clip. Used by ViViT, VideoMAE.

Tubelet embedding: Arnab et al. 2021 (ViViT, arXiv:2103.15691); Tong et al. 2022 (VideoMAE, arXiv:2203.12602).

Problem 1c: telling the model when each token is

- **Recall:** self-attention is permutation-invariant → position must be injected. For video we now need position on 3 axes: height, width, time
- **How production models do it:**
  - **M-RoPE** (Multimodal-RoPE, Qwen2-VL onward) for positional encoding : rotary position factorized over (t, h, w) —
    - Extends 2D-RoPE – which just takes token embedding, splits in half, and applies 1D RoPE based on *x* to 1st part, and 1D RoPE based on *y* to second part.
    - M-RoPE a token's adds t rotation: so split token embedding 3 ways – x, y, t – and rotates each accordingly
    - Positional embedding retains structure – so tokens/patches close in space have similar positional embeddings, and similar for those close temporally
    - Even better: M-RoPE can manage 1D, 2D and 2D+T RoPE (all needed for multimodal models) – e.g. text tokens have h=w=0, and just t changing; image only: h, w, but t = 0 and so on.
  - Absolute timestamps:
    - some models inject the real time (seconds) of each frame, so "when" questions can be answered in real units — essential for temporal grounding (see later)
- Why temporal position encoding matters more than for images:

  - for a static image, inaccurate patch embedding (within reason) degrades understanding;

  - for video, confusing frame order destroys causality ("picked up" vs "put down" etc)
- video token = embed(patch/tubelet content) + embed((where: h, w) and (when: t)).
- The transformer core is still completely unchanged.

Wang et al. 2024 (Qwen2-VL, arXiv:2409.12191) for M-RoPE over (t,h,w);

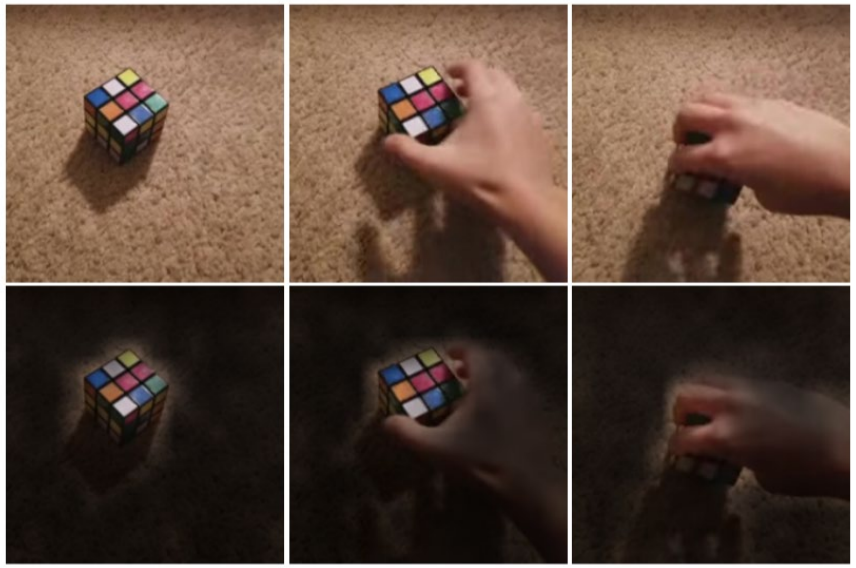
PART II

Managing attention

Why video breaks quadratic attention

- **Self-attention cost is $O(n^2)$ in sequence length n. For video, typically n = T · S:**
  - T = number of sampled frames, S = tokens per frame (e.g. 256–576)
  - 32 frames × 256 tokens = 8,192 tokens → $8{,}192^2 \approx 67$ MILLION attention scores per head, per layer
  - Double the frames → 4× the cost.

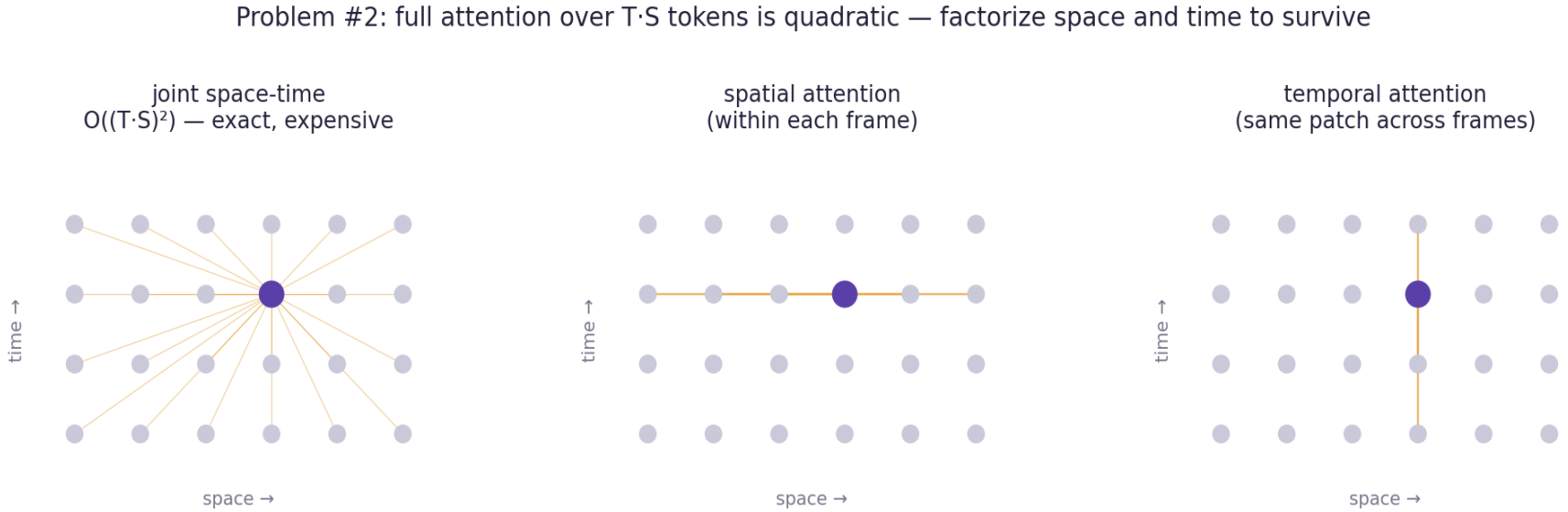
- **Three mitigation approaches used:**
  - Factorize attention — never attend over all T·S tokens at once
  - Pool / resample — shrink S or T with a learned or fixed reducer (a Perceiver type soln.)
  - Sparsify — attend to a subset (windows, keyframes) — particularly useful for long-video understanding





Cost analysis follows Vaswani et al. 2017; Arnab et al. 2021 §3 (factorization motivation).

Fix 1: factorized space-time attention



- Instead of one attention over all T·S tokens, pass token into temporal attention block, then spatial attention block:
  - attend across frames at each spatial position (time attention – patches only know about other patch at that location other masked)
  - then, take those output (time aware) tokens and attend within each frame only

    (space-attention across all patches in each frame allowed, other frame's patches masked out)
  - implemented via tensor reshaping to use standard transformer attention blocks
  - Final attention cost drops from $O((T \cdot S)^2)$ to $O(T^2 \cdot S + T \cdot S^2)$
- Trade-off: factorization can miss correlations that are jointly spatial-and-temporal — but usually a good trade-off

Arnab et al. 2021 (ViViT, arXiv:2103.15691); Bertasius et al. 2021 (TimeSformer, arXiv:2102.05095).

Fix 2: pool and resample the visual tokens

- Several production video-VLMs simply reduce the token count before the LLM sees it (before full attention computed)
  - Spatial pooling – reduce S
    - average/merge patches within a frame (e.g. the 2×2 patch-merge)
    - this reshapes ViT output tokens back to grid e.g. 196 → 14 x 14 and 2x2 tokens are averaged →

      7 x 7 tokens (only 25% of original # tokens)
  - Temporal pooling – reduce T
    - average token features across nearby frames, so same the corresponding token from each of the 7 x 7 token sequences above averages down to one token (over say sets of 4 consecutive frames)
  - Learned resampling:
    - a Perceiver Resampler / Q-Former maps a whole clip to a fixed number of tokens, regardless of length — the same "learned queries as a bottleneck" pattern, now over space-time
    - Recall Flamingo resampler: it was designed for this — many frames in, 64 tokens out
- The trade-off: every pool/resample step throws away temporal or spatial detail to fit the budget.
- Pool too much and you can no longer resolve detail e.g. "picked up vs put down" cannot be determined
- Design consequence: video-VLMs differ mostly in how they compress, not in the transformer core

Alayrac et al. 2022 (Perceiver Resampler); Li et al. 2023 (Q-Former); Maaz et al. 2023 (Video-ChatGPT, arXiv:2306.05424) for pooling in a video-VLM.

Video understanding: models and approaches

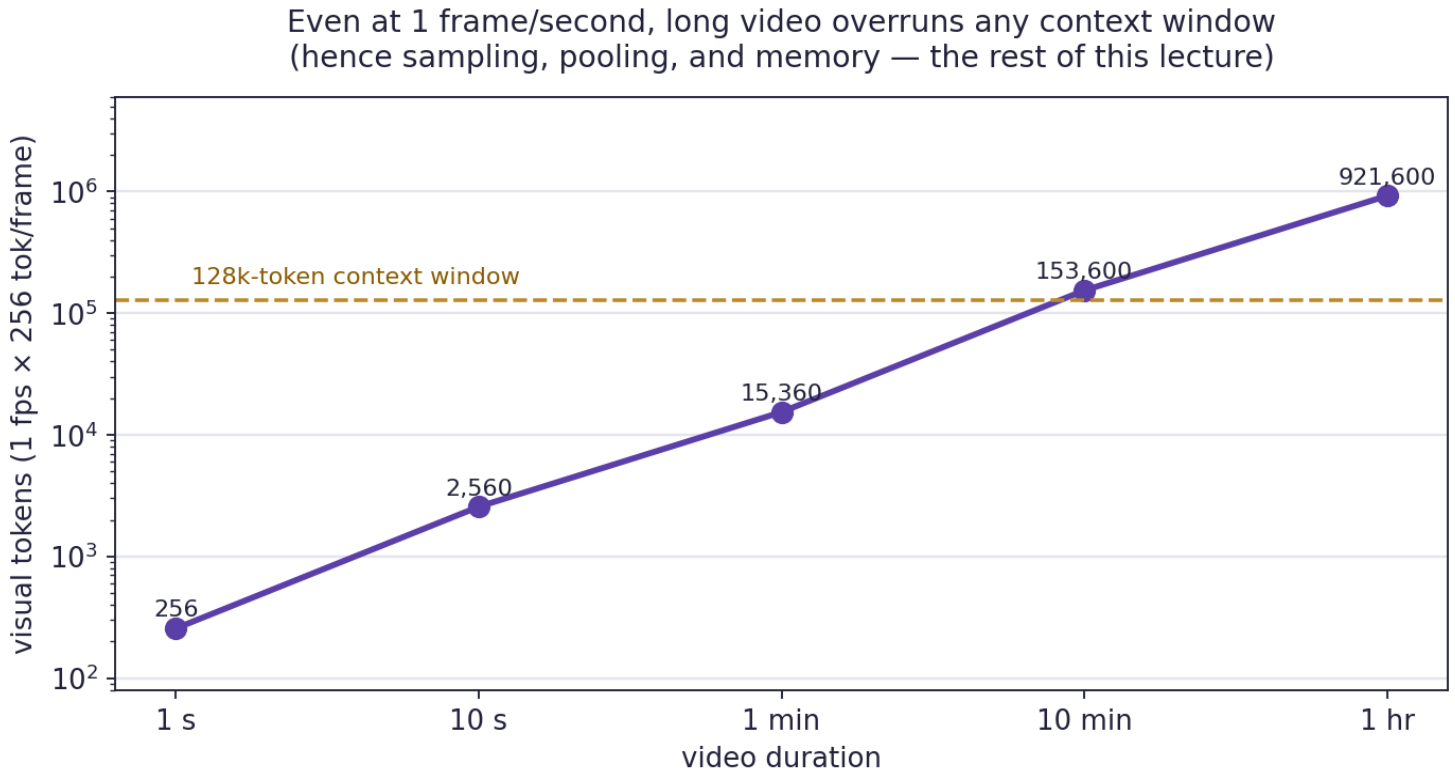
	video foundation models	video-LLM/VLMs (image-based VLM +time)
examples	ViViT, TimeSformer, VideoMAE, VideoMamba	Video-ChatGPT, Video-LLaVA, VideoLLaMA, Qwen2.5-VL, InternVL (video mode)
goal	strong video representations (classification, retrieval, video understanding, action recognition etc)	open-ended video chat, QA, captioning, grounding
time handling	tubelets + factorized attention, trained end-to-end	sample frames → per-frame ViT → compress → LLM reasons over time based on image tokens
relation to image understanding	extends ViT into 3D	extends align, bridge, instruction-tune with a temporal front-end

Lin et al. 2023 (Video-LLaVA, arXiv:2311.10122); Zhang et al. 2023 (VideoLLaMA, arXiv:2306.02858); Maaz et al. 2023; Arnab et al. 2021.

PART III

The long-video wall

The context wall



- Even at 1 frame/second, assuming only 256 tokens/frame, a 1-hour video needs ~920k tokens — exceeds most context windows
- But, 1 fps already loses most motion. Real understanding required more frames, not less
- So long-video understanding is fundamentally a compression and retrieval problem
- Four solutions/mitigations:
  - keyframe selection
  - memory
  - streaming
  - other modalities (audio/subtitles)

Scaling to long video – current approaches

strategy	idea	cost / limitation
keyframe selection	score frames for relevance (to the query, or by scene change) and keep only the informative ones	a bad selector discards the answer; query-aware selection needs the question up front
memory / hierarchical	summarize video chunks into compact memory tokens; reason over summaries ("read the video like chapters")	lossy compression; errors compound across chunks
streaming / online	process the video as it arrives, maintaining a running state — never hold it all at once	no global lookback unless stored; harder to answer "earlier" questions
other modalities	audio + subtitles carry huge information at ~zero visual-token cost — fuse them in	not always present/accurate; can leak answers (evaluation risk)

- None of these are solved. e.g. "Can a model watch a 2-hour film and answer a question about minute 12?" is a still open question.

He et al. 2024 (MA-LMM memory, arXiv:2404.05726); Korbar et al. 2024 (keyframe selection); Qwen2.5-VL §2 (audio/subtitle fusion, streaming).

Observation: the “cheapest” information is not visual

- A lecture video, a news clip, a tutorial: most of the meaning is in the speech, not the pixels
  - Subtitles / ASR (*) transcripts are already text

    → they cost the LLM almost nothing and slot straight into the token sequence
  - Audio events (a doorbell, a crash, applause) can help disambiguate visually ambiguous scenes
- This reframes many “video” benchmarks: a model can score well by reading subtitles and barely watching — this can give misleading impression of its capabilities, must design benchmark to stress test this (see later)
- Design pattern in current models: interleave [frame tokens] with [transcript text] on the **same** timeline, using timestamps to keep them aligned
- Interestingly “video understanding” in production is usually multimodal (vision + audio + text) understanding. Vision only understanding is much harder!

* (ASR = automatic speech recognition)

Multimodal fusion in video-LLMs: Zhang et al. 2023 (VideoLLaMA, audio branch); Qwen2.5-VL technical report.

PART IV

Tasks and evaluation

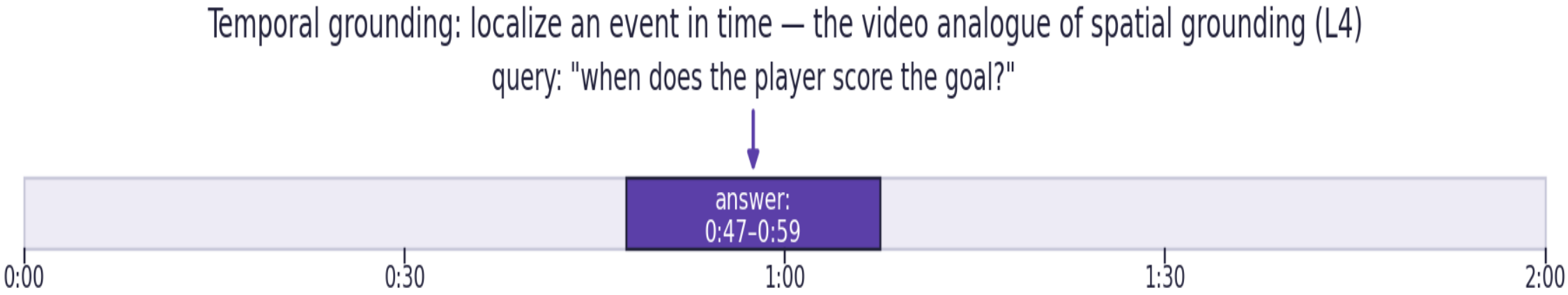
A taxonomy of video-understanding tasks

task	question form	how much temporal reasoning it needs
action recognition	"What action is this?" (classification)	low–medium — often a few frames suffice
video captioning	"Describe what happens."	medium — must order events, describe change
video QA	"Why did X do Y?" "What happened after Z?"	medium–high — causal, before/after, counting over time
temporal grounding	"When does event E occur?" → a time span	high — must localize in time, not just recognize
dense video captioning	caption every event with its timestamp	high — detect, localize, and describe, jointly

- Amount of temporal reasoning varies significantly - recognition needs little time-based reasoning while grounding needs a lot. A model may be good at one task but weak at a more temporally demanding task, hence benchmarks.

Task taxonomy after Xu et al. 2016 (MSR-VTT), Krishna et al. 2017 (dense captioning), Gao et al. 2017 (temporal grounding).

Example reasoning task: temporal grounding



- The direct analogue of spatial grounding: there, “where in the image?” → a box; here, “when in the video?” → a time span
- Output is just tokens: the model emits timestamps as text — enabled by the absolute-time positions noted earlier
- Why it’s hard: model can’t answer “when” by looking at one frame or reading a global summary — the model must align language to the timeline and examine image data at fine granularity

Figure: course original. Charades-STA (Gao et al. 2017); grounding-as-text in video-LLMs, e.g. Qwen2.5-VL, VTimeLLM (Huang et al. 2024, arXiv:2311.18445).

Benchmark issues: is the model using time at all?

- **The single-frame bias**
  - Many “video” questions are answerable from one well-chosen frame (“What sport is this?”). A model that ignores motion can top the leaderboard
  - Diagnostic: feed a single frame (or shuffle the frame order) — if the score barely drops, the benchmark wasn't testing temporal understanding
- **The subtitle/audio shortcut**
  - If transcripts are provided, a model can read rather than watch.
- **How benchmarks counter VLM shortcuts:**
  - Video-MME covers short to long clips and reports with/without subtitles, exposing the subtitle/audio shortcut
  - temporal-specific suites (e.g. TempCompass) build questions that a single frame or shuffled order cannot answer
- As before, when a video model “wins” at some task, ask — would it still win on 1 frame, on shuffled frames, or with the audio muted?

Fu et al. 2024 (Video-MME, arXiv:2405.21075); Liu et al. 2024 (TempCompass, arXiv:2403.00476); Lei et al. 2023 (single-frame bias, arXiv:2206.03428).

Common video VLM benchmarks

benchmark	probes	why it exists
MSR-VTT / ActivityNet-QA	open-ended video captioning and QA	classic general-purpose video QA
NExT-QA	causal and temporal QA (“why”, “before/after”)	forces reasoning beyond recognition
Charades-STA / ActivityNet-Captions	temporal grounding and dense captioning	localize events in time
Video-MME	short/medium/long, with and without subtitles	exposes the subtitle shortcut; tests length
TempCompass / Vinoground	temporal-only skills; frame-order sensitivity	built so a single/shuffled frame fails
EgoSchema / LongVideoBench	long-form understanding (minutes+)	tests the long video context wall

- Choose benchmarks which reflect your application priorities.

Xu et al. 2016; Yu et al. 2019 (ActivityNet-QA); Xiao et al. 2021 (NExT-QA); Gao et al. 2017; Fu et al. 2024; Liu et al. 2024 (TempCompass); Mangalam et al. 2023 (EgoSchema).

Summary

Tokenizing

A video token = content (patch or tubelet) + spatial position (h,w) + temporal position (t). Tubelets encode in short-term motion; M-RoPE supplies the third axis. The transformer core is unchanged.

Attention

$n = T \cdot S$ makes attention $O((T \cdot S)^2)$. Preserve it by factorizing space vs time, and by pooling/resampling tokens before the LLM (Perceiver/Q-Former, see earlier). Every compression trades temporal detail for budget.

Long video

Hours overrun common context window (1 hr ≈ 920k tokens at 1 fps). Keyframe selection, memory, streaming, and cheap audio/subtitle tokens push the wall back — but hour-scale understanding is an open problem.

Evaluation

Ask whether the model used time at all: single-frame bias and the subtitle shortcut flatter models. Temporal grounding is the acid test; Video-MME / TempCompass are built to defeat shortcuts.

Resources

- **Papers/references**
  - Arnab et al. (2021). ViViT: A Video Vision Transformer. arXiv:2103.15691 · Bertasius et al. (2021). TimeSformer. arXiv:2102.05095 · Tong et al. (2022). VideoMAE. arXiv:2203.12602
  - Maaz et al. (2023). Video-ChatGPT. arXiv:2306.05424 · Lin et al. (2023). Video-LLaVA. arXiv:2311.10122 · Zhang et al. (2023). VideoLLaMA. arXiv:2306.02858 · Wang et al. (2024). Qwen2-VL (M-RoPE). arXiv:2409.12191
  - He et al. (2024). MA-LMM. arXiv:2404.05726 · Huang et al. (2024). VTimeLLM. arXiv:2311.18445 · Mangalam et al. (2023). EgoSchema. arXiv:2308.09126
  - Fu et al. (2024). Video-MME. arXiv:2405.21075 · Liu et al. (2024). TempCompass. arXiv:2403.00476 · Xiao et al. (2021). NExT-QA. arXiv:2105.08276 · Gao et al. (2017). Charades-STA/TALL. arXiv:1705.02101
- **Code to try**
  - Run a small video-LLM from the HF Hub on a short clip; vary the number of sampled frames and watch the answer change